



Sintaxis

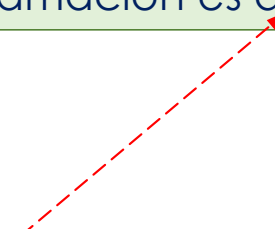
Fundamentos de Lenguajes de Programación – IIC2560

Dar una descripción concisa y al mismo tiempo inteligible de un lenguaje de programación es difícil pero esencial



Dar una descripción concisa y al mismo tiempo inteligible de un lenguaje de programación es difícil pero esencial

P.ej., diversidad de personas que deben entender la descripción



Dar una descripción concisa y al mismo tiempo inteligible de un lenguaje de programación es difícil pero esencial

P.ej., diversidad de personas que deben entender la descripción

Implementadores: Deben poder determinar cómo están formadas las expresiones, sentencias y unidades de un programa, y su efecto intencionado cuando son ejecutadas

Usuarios: Deben poder determinar cómo codificar soluciones de software a partir de un manual de referencia del lenguaje

Estudio de los lenguajes de programación:

- estudio de la **sintaxis**
- estudio de la **semántica**



Estudio de los lenguajes de programación:

- estudio de la sintaxis
- estudio de la semántica

La **forma** de las expresiones, sentencias y unidades de un programa

El **significado** de esas expresiones, sentencias y unidades

Estudio de los lenguajes de programación:

- estudio de la sintaxis
- estudio de la semántica

La forma de las expresiones, sentencias y unidades de un programa

P.ej., la sintaxis de la sentencia **while** en Java es
`while (boolean_expr) statement`

El significado de esas expresiones, sentencias y unidades

Estudio de los lenguajes de programación:

- estudio de la sintaxis
- estudio de la semántica

La forma de las expresiones, sentencias y unidades de un programa

P.ej., la sintaxis de la sentencia **while** en Java es
`while (boolean_expr) statement`

El significado de esas expresiones, sentencias y unidades

P.ej., la semántica de la sentencia **while** es que
Cuando el valor actual de *boolean_expr* es verdadero, la sentencia *statement*, subordinada, es ejecutada ... y el control vuelve (implícitamente) a *boolean_expr* para repetir el proceso.
Si *boolean_expr* es falsa, entonces el control es transferido a la sentencia que sigue al **while**

Un **lenguaje** es un conjunto de strings de caracteres de algún alfabeto

Natural o artificial

Sentencias o declaraciones

Un lenguaje es un conjunto de strings de caracteres de algún alfabeto

Un lenguaje es un conjunto de **strings** de caracteres de algún alfabeto

Las **reglas de sintaxis** del lenguaje especifican cuáles strings están en el lenguaje

Natural o artificial

Sentencias o declaraciones

Un lenguaje es un conjunto de strings de caracteres de algún alfabeto

Las reglas de sintaxis del lenguaje especifican cuáles strings están en el lenguaje

La forma de las expresiones, sentencias y unidades de un programa

Típicamente, la descripción de la sintaxis de un lenguaje de programación no incluye la descripción de los **lexemas**

Típicamente, la descripción de la sintaxis de un lenguaje de programación no incluye la descripción de los **lexemas**

P.ej., lexemas y tokens de la sentencia

```
index = 2 * count + 17;
```

Lexemas	Tokens
index	identificador
=	signo_igual
2	literal_int
*	op_mult
count	identificador
+	op_sum
17	literal_int
;	punto_y_coma

Unidades sintácticas del nivel más bajo:

- p.ej., literales numéricos, operadores, palabras especiales

Un programa puede ser visto como un string de lexemas, en lugar de caracteres

Se particionan en grupos:

- p.ej., los nombres de variables, métodos, clases, etc. forman un grupo

Cada grupo es representado por un nombre o **token**

- p.ej., el grupo anterior es llamado *identificadores*

N. Chomsky describió cuatro clases de **gramáticas** (1956, 59), que definen cuatro clases de lenguajes ... dos de los cuales se usan para describir la sintaxis de lenguajes de programación:

- gramáticas *libres de contexto* → la sintaxis de lenguajes de programación completos
- gramáticas *regulares* → las formas de los tokens

N. Chomsky describió cuatro clases de **gramáticas** (1956, 59), que definen cuatro clases de lenguajes ... dos de los cuales se usan para describir la sintaxis de lenguajes de programación:

- gramáticas *libres de contexto* → la sintaxis de lenguajes de programación completos
- gramáticas *regulares* → las formas de los tokens

Mecanismos formales de generación de lenguajes ... usados para describir la sintaxis de lenguajes de programación

J. Backus, del grupo ACM-GAMM, presentó un paper(1959) describiendo el lenguaje ALGOL 58 e introduciendo una notación formal para especificar la sintaxis de lenguajes de programación

La notación, levemente modificada por P. Naur para describir ALGOL 60 (1960)

... se conoce como **Backus-Naur Form** o **BNF**

N. Chomsky describió cuatro clases de **gramáticas** (1956, 59), que definen cuatro clases de lenguajes ... dos de los cuales se usan para describir la sintaxis de lenguajes de programación:

- gramáticas *libres de contexto* → la sintaxis de lenguajes de programación completos
- gramáticas *regulares* → las formas de los tokens

J. Backus, del grupo ACM-GAMM, presentó un paper(1959) describiendo el lenguaje ALGOL 58 e introduciendo una notación formal para especificar la sintaxis de lenguajes de programación

La notación, levemente modificada por P. Naur para describir ALGOL 60 (1960)

... se conoce como **Backus-Naur Form** o **BNF**

BNF es casi idéntico a las gramáticas libres de contexto

Un **metalenguaje** es un lenguaje que es usado para describir otro lenguaje

Un **metalenguaje** es un lenguaje que es usado para describir otro lenguaje

BNF es un metalenguaje para lenguajes de programación

BNF usa abstracciones para las estructuras sintácticas

- P.ej., la definición de la sentencia de asignación en Java está dada por la siguiente **regla** o **producción**:

$$\langle \text{assign} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expression} \rangle$$

Un **metalenguaje** es un lenguaje que es usado para describir otro lenguaje

BNF es un metalenguaje para lenguajes de programación

BNF usa abstracciones para las estructuras sintácticas

- P.ej., la definición de la sentencia de asignación en Java está dada por la siguiente **regla** o **producción**:

`<assign> → <var> = <expression>`

El lado izquierdo de la flecha (**LHS**) es la abstracción que está siendo definida

Abstracciones que también deben ser definidas

El lado derecho (**RHS**) de la flecha es la definición del LHS

Abstracción que representa la sentencia de asignación

La **regla** o **producción**

$\langle \text{assign} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expression} \rangle$

... especifica que la abstracción $\langle \text{assign} \rangle$ es definida como

... una instancia de la abstracción $\langle \text{var} \rangle$

... seguida por el lexema =

... seguido por una instancia de la abstracción $\langle \text{expression} \rangle$

P.ej., la sentencia

`total = subtotal1 + subtotal2`

Símbolos **no terminales** (las abstracciones)

Símbolos **terminales**
(los lexemas y tokens)

La **regla o producción**

$\langle \text{assign} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expression} \rangle$

... especifica que la abstracción $\langle \text{assign} \rangle$ es definida como

... una instancia de la abstracción $\langle \text{var} \rangle$

... seguida por el lexema =

... seguido por una instancia de la abstracción $\langle \text{expression} \rangle$

P.ej., la sentencia

`total = subtotal1 + subtotal2`

$\langle \text{var} \rangle$

=

$\langle \text{expression} \rangle$

Los (símbolos) **no terminales** pueden tener varias definiciones:
varias formas sintácticas

- P.ej., la sentencia **if** de Java

$\langle \text{if_stmt} \rangle \rightarrow \text{if} (\langle \text{logic_expr} \rangle) \langle \text{stmt} \rangle$

$\langle \text{if_stmt} \rangle \rightarrow \text{if} (\langle \text{logic_expr} \rangle) \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle$

or with the rule

$\langle \text{if_stmt} \rangle \rightarrow \text{if} (\langle \text{logic_expr} \rangle) \langle \text{stmt} \rangle$

| $\text{if} (\langle \text{logic_expr} \rangle) \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle$

Los (símbolos) **no terminales** pueden tener varias definiciones:
varias formas sintácticas

- P.ej., la sentencia **if** de Java

Una forma de especificar
dos (o más) definiciones del
no terminal `<if_stmt>`: una
regla para cada definición

`<if_stmt> → if ( <logic_expr> ) <stmt>`
`<if_stmt> → if ( <logic_expr> ) <stmt> else <stmt>`

or with the rule

`<if_stmt> → if ( <logic_expr> ) <stmt>`
`| if ( <logic_expr> ) <stmt> else <stmt>`

Otra forma de especificar
dos (o más) definiciones
del no terminal `<if_stmt>`:

... las diferentes definiciones
son separadas por el símbolo
“|” que representa el **○** lógico

Las listas se describen recursivamente

Una regla es **recursiva** si su LHS aparece en su RHS

- P.ej., la siguiente regla describe una lista de identificadores

$$\begin{aligned} \langle \text{ident_list} \rangle &\rightarrow \text{identifier} \\ &| \text{identifier}, \langle \text{ident_list} \rangle \end{aligned}$$

Las listas se describen recursivamente

Una regla es **recursiva** si su LHS aparece en su RHS

- P.ej., la siguiente regla describe una lista de identificadores

$\langle \text{ident_list} \rangle \rightarrow \text{identifier}$
| $\text{identifier}, \langle \text{ident_list} \rangle$

$\langle \text{ident_list} \rangle$ es ya sea un
único token (*identifier*) ...

... o un *identifier* seguido por una
coma y otra instancia de $\langle \text{ident_list} \rangle$

Una gramática es un dispositivo generativo para definir lenguajes

Las sentencias del lenguaje son generadas a través de una sucesión de aplicaciones de las reglas —una **derivación**— a partir de un no terminal especial —el **símbolo de partida**:

- en una gramática para un lenguaje completo el símbolo de partida —típicamente, `<program>` — representa un programa completo

P.ej., consideremos la siguiente gramática, que describe un lenguaje que tiene un solo tipo de sentencias: asignaciones

```
<program> → begin <stmt_list> end  
<stmt_list> → <stmt>  
               | <stmt> ; <stmt_list>  
<stmt> → <var> = <expression>  
<var> → A | B | C  
<expression> → <var> + <var>  
               | <var> - <var>  
               | <var>
```

P.ej., consideremos la siguiente gramática, que describe un lenguaje que tiene un solo tipo de sentencias: asignaciones

Un programa consiste en la palabra especial **begin**, seguido por una lista de sentencias separadas por punto y comas, seguida por la palabra especial **end**

Una expresión es ya sea dos variables separadas por el operador **+** o el operador **-** ... o una única variable

```
<program> → begin <stmt_list> end
<stmt_list> → <stmt>
               | <stmt> ; <stmt_list>
<stmt> → <var> = <expression>
<var> → A | B | C
<expression> → <var> + <var>
               | <var> - <var>
               | <var>
```

Los únicos nombres de variables son **A**, **B** y **C**

P.ej., derivación de un programa en el lenguaje definido por la gramática anterior

```
<program> => begin <stmt_list> end  
=> begin <stmt> ; <stmt_list> end  
=> begin <var> = <expression> ; <stmt_list> end  
=> begin A = <expression> ; <stmt_list> end  
=> begin A = <var> + <var> ; <stmt_list> end  
=> begin A = B + <var> ; <stmt_list> end  
=> begin A = B + C ; <stmt_list> end  
=> begin A = B + ; <stmt> end  
=> begin A = B + C ; <var> = <expression> end  
=> begin A = B + C ; B = <expression> end  
=> begin A = B + C ; B = <var> end  
=> begin A = B + C ; B = C end
```

P.ej., derivación de un programa en el lenguaje definido por la gramática anterior

La derivación comienza con el símbolo de partida

$\langle \text{program} \rangle \rightarrow \text{begin } \langle \text{stmt_list} \rangle \text{ end}$
 $\langle \text{stmt_list} \rangle \rightarrow \langle \text{stmt} \rangle$
 $\quad \quad \quad | \langle \text{stmt} \rangle ; \langle \text{stmt_list} \rangle$
 $\langle \text{stmt} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expression} \rangle$
 $\quad \quad \quad \vdots$
 $\langle \text{var} \rangle \rightarrow A \mid B \mid C$
 $\langle \text{expression} \rangle \rightarrow \langle \text{var} \rangle + \langle \text{var} \rangle$
 $\quad \quad \quad | \langle \text{var} \rangle - \langle \text{var} \rangle$
 $\quad \quad \quad | \langle \text{var} \rangle$

$\langle \text{program} \rangle \Rightarrow \text{begin } \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } \langle \text{stmt} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } \langle \text{var} \rangle = \langle \text{expression} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = \langle \text{expression} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = \langle \text{var} \rangle + \langle \text{var} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + \langle \text{var} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + ; \langle \text{stmt} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; \langle \text{var} \rangle = \langle \text{expression} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; B = \langle \text{expression} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; B = \langle \text{var} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; B = C \text{ end}$

P.ej., derivación de un programa en el lenguaje definido por la gramática anterior

La derivación comienza con el símbolo de partida

"deriva en"

Cada string sucesivo en la secuencia es derivado del string anterior reemplazando uno de los no terminales por una de las definiciones de ese no terminal

Este último string es el programa:

- no tiene no terminales

```

<program> => begin <stmt_list> end
=> begin <stmt> ; <stmt_list> end
=> begin <var> = <expression> ; <stmt_list> end
=> begin A = <expression> ; <stmt_list> end
=> begin A = <var> + <var> ; <stmt_list> end
=> begin A = B + <var> ; <stmt_list> end
=> begin A = B + C ; <stmt_list> end
=> begin A = B + ; <stmt> end
=> begin A = B + C ; <var> = <expression> end
=> begin A = B + C ; B = <expression> end
=> begin A = B + C ; B = <var> end
=> begin A = B + C ; B = C end
  
```

```
<program> => begin <stmt_list> end  
=> begin <stmt> ; <stmt_list> end  
=> begin <var> = <expression> ; <stmt_list> end  
=> begin A = <expression> ; <stmt_list> end  
=> begin A = <var> + <var> ; <stmt_list> end  
=> begin A = B + <var> ; <stmt_list> end  
=> begin A = B + C ; <stmt_list> end  
=> begin A = B + ; <stmt> end  
=> begin A = B + C ; <var> = <expression> end  
=> begin A = B + C ; B = <expression> end  
=> begin A = B + C ; B = <var> end  
=> begin A = B + C ; B = C end
```

Cada string, incluido `<program>`, se llama una **forma sentencial**

La derivación sigue hasta que la forma sentencial derivada no contenga no terminales ...

... es decir, contenga sólo terminales, o lexemas:

- esta es la sentencia generada

Esta derivación es **leftmost** (más a la izquierda):

- el no terminal reemplazado es siempre el que está más a la izquierda en la forma sentencial anterior

$\langle \text{program} \rangle \rightarrow \text{begin } \langle \text{stmt_list} \rangle \text{ end}$
 $\langle \text{stmt_list} \rangle \rightarrow \langle \text{stmt} \rangle$
 | $\langle \text{stmt} \rangle ; \langle \text{stmt_list} \rangle$
 $\langle \text{stmt} \rangle \rightarrow \langle \text{var} \rangle = \langle \text{expression} \rangle$
 $\langle \text{var} \rangle \rightarrow A \mid B \mid C$
 $\langle \text{expression} \rangle \rightarrow \langle \text{var} \rangle + \langle \text{var} \rangle$
 | $\langle \text{var} \rangle - \langle \text{var} \rangle$
 | $\langle \text{var} \rangle$

$\langle \text{program} \rangle \Rightarrow \text{begin } \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } \langle \text{stmt} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } \langle \text{var} \rangle = \langle \text{expression} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = \langle \text{expression} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = \langle \text{var} \rangle + \langle \text{var} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + \langle \text{var} \rangle ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; \langle \text{stmt_list} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + ; \langle \text{stmt} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; \langle \text{var} \rangle = \langle \text{expression} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; B = \langle \text{expression} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; B = \langle \text{var} \rangle \text{ end}$
 $\Rightarrow \text{begin } A = B + C ; B = C \text{ end}$

Una derivación también puede ser *rightmost* o seguir un orden que no sea *leftmost* ni *rightmost*:

- el orden de la derivación no tiene efecto en el lenguaje generado por una gramática

Una derivación también puede ser *rightmost* o seguir un orden que no sea *leftmost* ni *rightmost*:

- el orden de la derivación no tiene efecto en el lenguaje generado por una gramática

Al elegir RHSs alternativos con los cuales reemplazar no terminales en una derivación, se pueden generar diferentes sentencias del lenguaje:

- si elegimos exhaustivamente todas las combinaciones de posibilidades, entonces podemos generar el lenguaje completo

Una derivación también puede ser *rightmost* o seguir un orden que no sea *leftmost* ni *rightmost*:

- el orden de la derivación no tiene efecto en el lenguaje generado por una gramática

Al elegir RHSs alternativos con los cuales reemplazar no terminales en una derivación, se pueden generar diferentes sentencias del lenguaje:

- si elegimos exhaustivamente todas las combinaciones de posibilidades, entonces podemos generar el lenguaje completo

El lenguaje definido por la gramática anterior, como muchos otros, es *infinito*:

- no podemos generar todas las sentencias del lenguaje en tiempo finito

La siguiente gramática describe asignaciones en un lenguaje de programación típico

$$\begin{aligned}\langle \text{assign} \rangle &\rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle \\ \langle \text{id} \rangle &\rightarrow A \mid B \mid C \\ \langle \text{expr} \rangle &\rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle \\ &\quad \mid \langle \text{id} \rangle * \langle \text{expr} \rangle \\ &\quad \mid (\langle \text{expr} \rangle) \\ &\quad \mid \langle \text{id} \rangle\end{aligned}$$

Asignaciones cuyo lado derecho (a la derecha del signo “=”) son expresiones aritméticas con paréntesis y con las operaciones multiplicación y suma

... y consideremos la derivación de una asignación particular

P.ej., la asignación

A = B * (A + C)

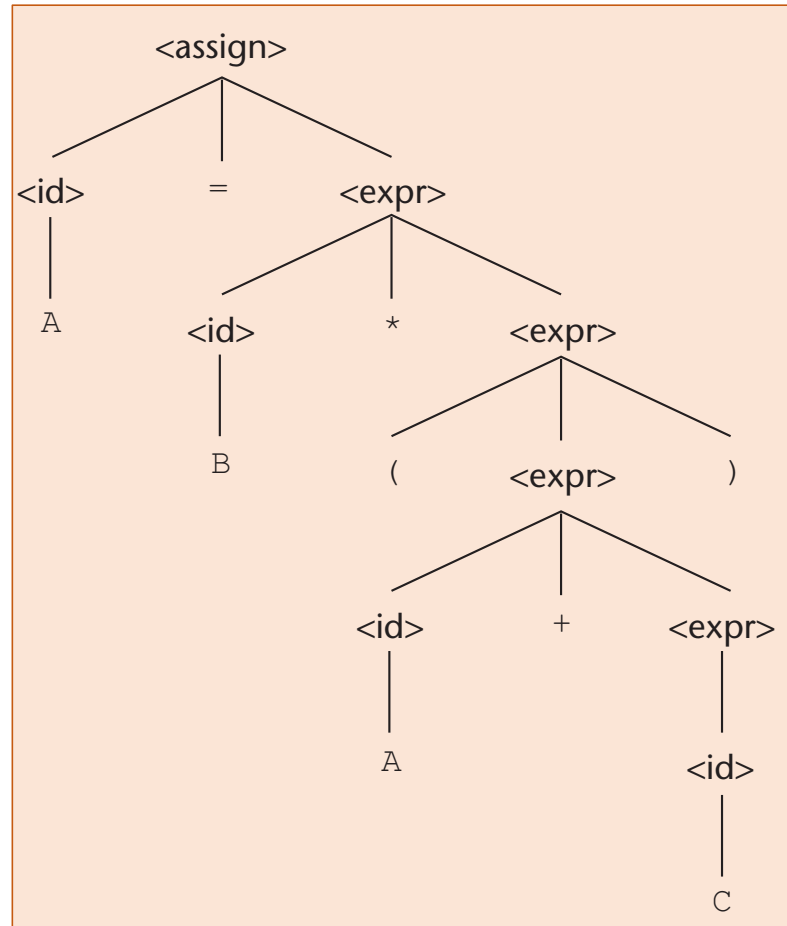
... es generada por esta derivación *leftmost*

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$
 $\langle \text{id} \rangle \rightarrow A \mid B \mid C$
 $\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle$
 $\quad \mid \langle \text{id} \rangle * \langle \text{expr} \rangle$
 $\quad \mid (\langle \text{expr} \rangle)$
 $\quad \mid \langle \text{id} \rangle$

$\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$
 $\Rightarrow A = \langle \text{expr} \rangle$
 $\Rightarrow A = \langle \text{id} \rangle * \langle \text{expr} \rangle$
 $\Rightarrow A = B * \langle \text{expr} \rangle$
 $\Rightarrow A = B * (\langle \text{expr} \rangle)$
 $\Rightarrow A = B * (\langle \text{id} \rangle + \langle \text{expr} \rangle)$
 $\Rightarrow A = B * (A + \langle \text{expr} \rangle)$
 $\Rightarrow A = B * (A + \langle \text{id} \rangle)$
 $\Rightarrow A = B * (A + C)$

Las gramáticas también describen la jerarquía de la estructura sintáctica —el **árbol de parse**— de las sentencias del lenguaje

$\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$
 $\Rightarrow A = \langle \text{expr} \rangle$
 $\Rightarrow A = \langle \text{id} \rangle * \langle \text{expr} \rangle$
 $\Rightarrow A = B * \langle \text{expr} \rangle$
 $\Rightarrow A = B * (\langle \text{expr} \rangle)$
 $\Rightarrow A = B * (\langle \text{id} \rangle + \langle \text{expr} \rangle)$
 $\Rightarrow A = B * (A + \langle \text{expr} \rangle)$
 $\Rightarrow A = B * (A + \langle \text{id} \rangle)$
 $\Rightarrow A = B * (A + C)$



Las gramáticas también describen la jerarquía de la estructura sintáctica —el **árbol de parse**— de las sentencias del lenguaje

```

<assign> => <id> = <expr>
=> A = <expr>
=> A = <id> * <expr>
=> A = B * <expr>
=> A = B * ( <expr> )
=> A = B * ( <id> + <expr> )
=> A = B * ( A + <expr> )
=> A = B * ( A + <id> )
=> A = B * ( A + C )
  
```

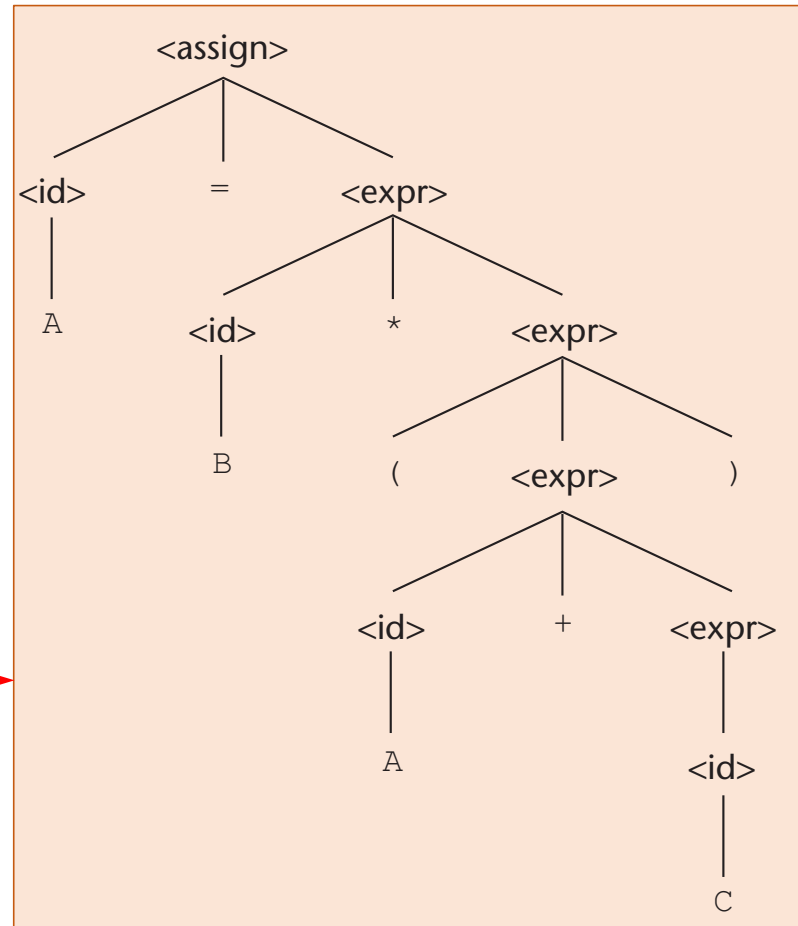
P.ej., árbol de *parse* de

A = B * (A + C)

Cada nodo interno del árbol es etiquetado con un no terminal

... y cada hoja es etiquetada con un terminal

... y cada subárbol describe una instancia de una abstracción en la sentencia



La siguiente gramática —que también describe asignaciones— es levemente diferente de la gramática de la diap. 39

$$\begin{aligned}\langle \text{assign} \rangle &\rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle \\ \langle \text{id} \rangle &\rightarrow A \mid B \mid C \\ \langle \text{expr} \rangle &\rightarrow \langle \text{expr} \rangle + \langle \text{expr} \rangle \\ &\quad \mid \langle \text{expr} \rangle * \langle \text{expr} \rangle \\ &\quad \mid (\langle \text{expr} \rangle) \\ &\quad \mid \langle \text{id} \rangle\end{aligned}$$
$$\begin{aligned}\langle \text{assign} \rangle &\rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle \\ \langle \text{id} \rangle &\rightarrow A \mid B \mid C \\ \langle \text{expr} \rangle &\rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle \\ &\quad \mid \langle \text{id} \rangle * \langle \text{expr} \rangle \\ &\quad \mid (\langle \text{expr} \rangle) \\ &\quad \mid \langle \text{id} \rangle\end{aligned}$$

La siguiente gramática —que también describe asignaciones— es levemente diferente de la gramática de la diap. 39

$$\begin{aligned}\langle \text{assign} \rangle &\rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle \\ \langle \text{id} \rangle &\rightarrow A \mid B \mid C \\ \langle \text{expr} \rangle &\rightarrow \langle \text{expr} \rangle + \langle \text{expr} \rangle \\ &\quad \mid \langle \text{expr} \rangle * \langle \text{expr} \rangle \\ &\quad \mid (\langle \text{expr} \rangle) \\ &\quad \mid \langle \text{id} \rangle\end{aligned}$$
$$\begin{aligned}\langle \text{assign} \rangle &\rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle \\ \langle \text{id} \rangle &\rightarrow A \mid B \mid C \\ \langle \text{expr} \rangle &\rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle \\ &\quad \mid \langle \text{id} \rangle * \langle \text{expr} \rangle \\ &\quad \mid (\langle \text{expr} \rangle) \\ &\quad \mid \langle \text{id} \rangle\end{aligned}$$

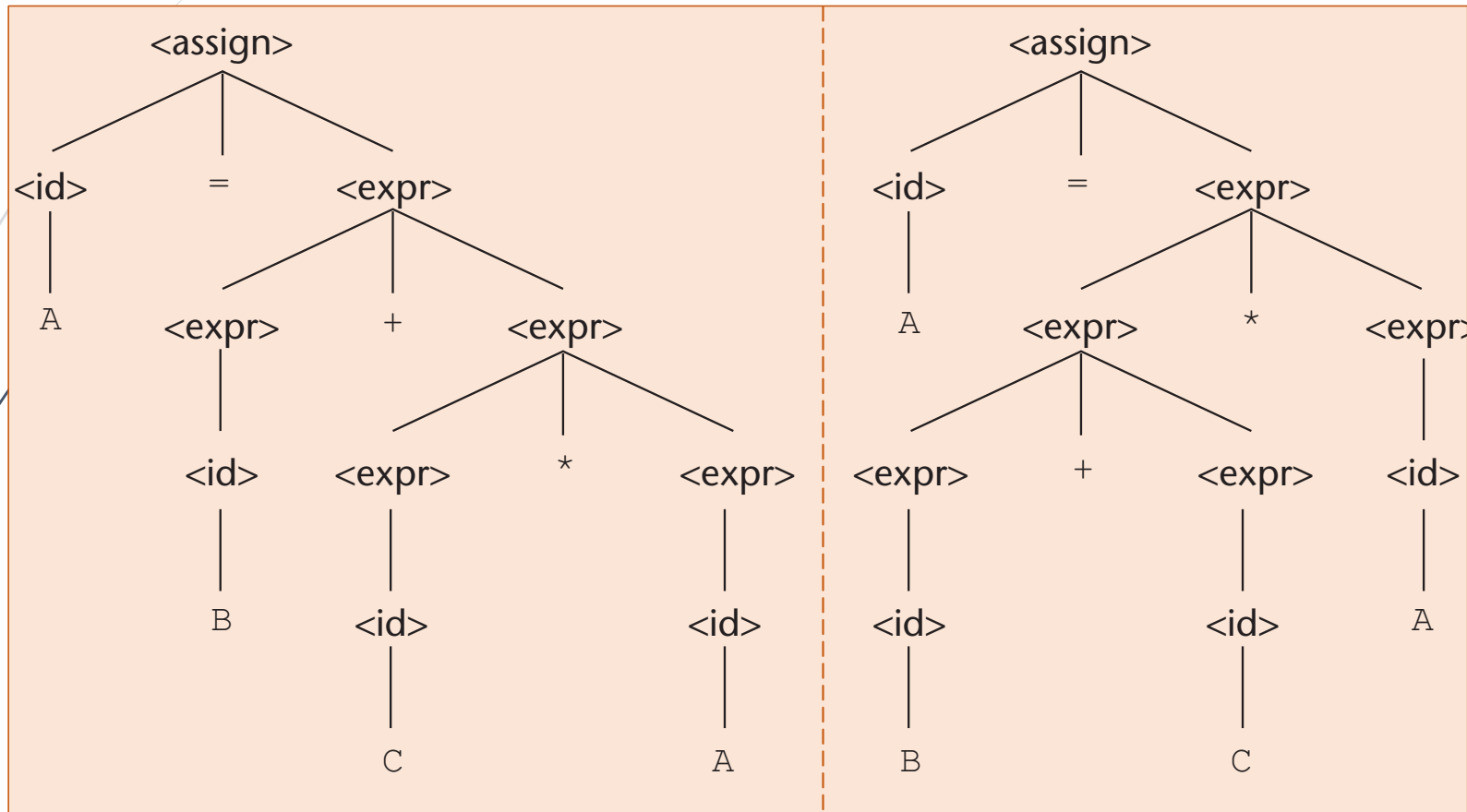
Puede generar asignaciones *para las cuales hay dos o más árboles de parse distintos*:

- la gramática es **ambigua** (específicamente, porque el árbol de *parse* para una expresión, $\langle \text{expr} \rangle$, puede crecer tanto por la izquierda como por la derecha)

P.ej., la gramática anterior es ambigua porque la sentencia

$$A = B + C * A$$

tiene dos árboles de parse distintos



La ambigüedad sintáctica de las estructuras de un lenguaje es un problema → los compiladores basan la semántica de esas estructuras en sus formas sintácticas:

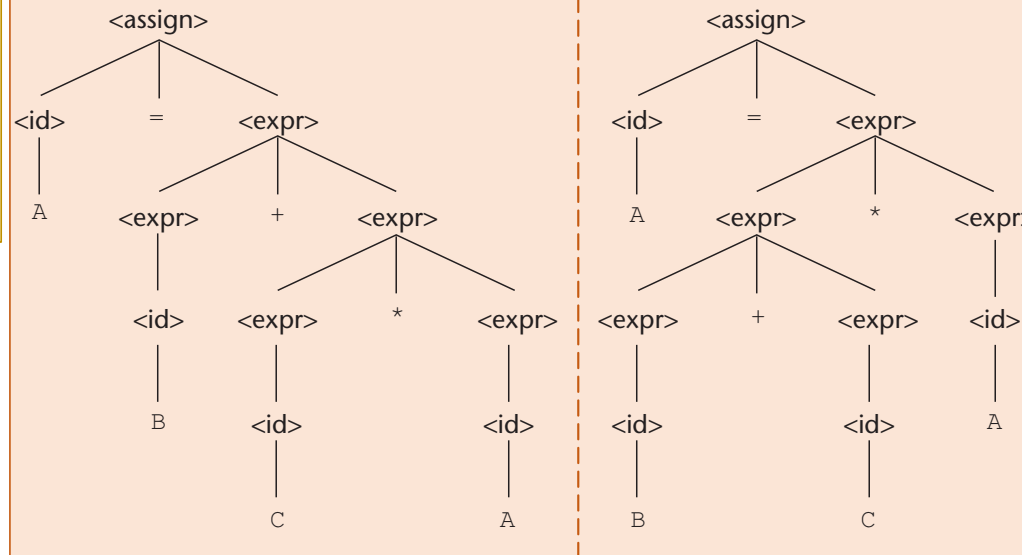
- el compilador decide qué código tiene que generar para una sentencia revisando el árbol de *parse* de ésta
- si una estructura tiene más de un árbol de *parse*, entonces su significado no puede ser determinado de manera única

en lenguaje
assembly del
computador

La ambigüedad sintáctica de las estructuras de un lenguaje es un problema → los compiladores basan la semántica de esas estructuras en sus formas sintácticas:

- el compilador decide qué código tiene que generar para una sentencia revisando el árbol de *parse* de ésta
- si una estructura tiene más de un árbol de *parse*, entonces su significado no puede ser determinado de manera única

P.ej., $A = B + C * A$
es interpretada como
 $A = B + (C * A)$
si se usa este árbol ...



... y como
 $A = (B + C) * A$
si se usa este otro árbol

La expresión $B + C * A$ tiene dos operadores diferentes ... y su valor depende del orden de evaluación de las operaciones —¿sumamos primero y después multiplicamos, o vice versa?

Resolvemos esta ambigüedad semántica asignando diferentes niveles de **precedencia** a los operadores:

- p.ej., si $*$ tiene mayor precedencia que $+$, entonces la multiplicación se hace primero ...

La expresión $B + C * A$ tiene dos operadores diferentes ... y su valor depende del orden de evaluación de las operaciones —¿sumamos primero y después multiplicamos, o vice versa?

Resolvemos esta ambigüedad semántica asignando diferentes niveles de **precedencia** a los operadores:

- p.ej., si $*$ tiene mayor precedencia que $+$, entonces la multiplicación se hace primero ...

... sin importar el orden de aparición de los operadores en la expresión

P.ej., si $A = 0$, entonces, si primero sumamos, el valor de la expresión es 0 ... pero si primero multiplicamos, el valor es B

Asignada (arbitrariamente) por el diseñador del lenguaje

Una gramática puede describir una estructura sintáctica de tal manera que (parte de) su significado pueda determinarse a partir de su árbol de *parse*:

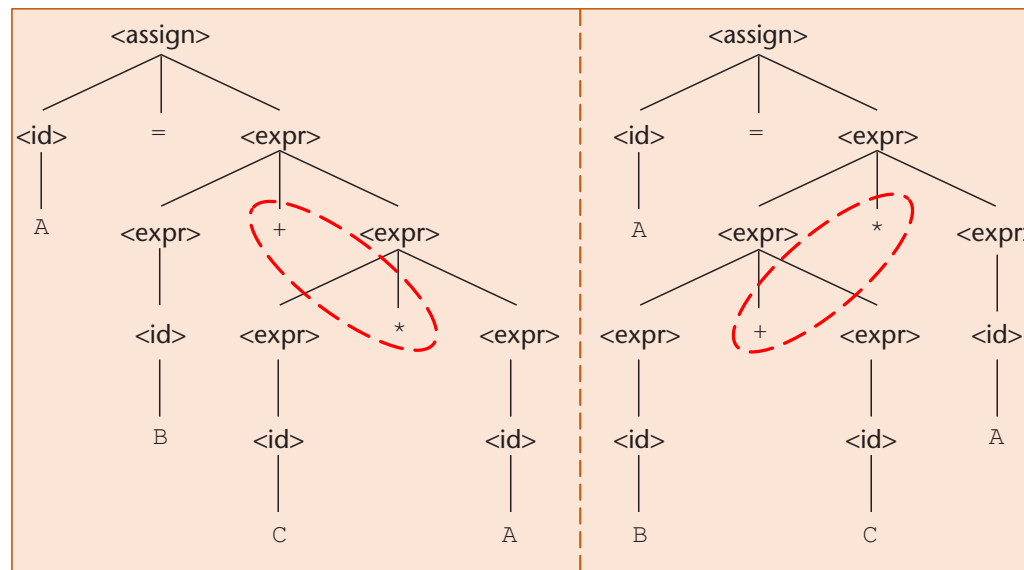
- p.ej., si un operador en una expresión aritmética es generado más abajo en el árbol de *parse* —por lo que debe ser evaluado primero— significa que tiene precedencia sobre un operador producido más arriba

Una gramática puede describir una estructura sintáctica de tal manera que (parte de) su significado pueda determinarse a partir de su árbol de *parse*:

- p.ej., si un operador en una expresión aritmética es generado más abajo en el árbol de *parse* —por lo que debe ser evaluado primero— significa que tiene precedencia sobre un operador producido más arriba

Estos dos árboles dan información de precedencia contradictoria

P.ej., $A = B + C * A$
es interpretada como
 $A = B + (C * A)$
si se usa este árbol ...



... y como
 $A = (B + C) * A$
si se usa este otro árbol

La gramática de la diap. 39 no es ambigua:

... pero la precedencia de los operadores en una expresión depende de la posición relativa del operador en la expresión (y no del operador mismo):

- el operador de más a la derecha en la expresión es producido más abajo en el árbol de *parse*
- los otros operadores aparecen progresivamente más arriba a medida que nos movemos hacia la izquierda en la expresión

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> + <expr>
        | <id> * <expr>
        | ( <expr> )
        | <id>
```

Queremos una gramática que

- no sea ambigua
- especifique una precedencia consistente para $+$ y $*$, sin importar el orden en que éstos aparezcan en una expresión

Queremos una gramática que

- no sea ambigua
- especifique una precedencia consistente para + y *, sin importar el orden en que éstos aparezcan en una expresión

Nuevas reglas

$$\begin{aligned} \langle \text{assign} \rangle &\rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle \\ \langle \text{id} \rangle &\rightarrow A \mid B \mid C \\ \langle \text{expr} \rangle &\rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \\ &\quad \mid \langle \text{term} \rangle \\ \langle \text{term} \rangle &\rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \\ &\quad \mid \langle \text{factor} \rangle \\ \langle \text{factor} \rangle &\rightarrow (\langle \text{expr} \rangle) \\ &\quad \mid \langle \text{id} \rangle \end{aligned}$$

Símbolos no terminales diferentes (adicionales) para representar los operandos de los operadores que tienen diferente precedencia

$\langle \text{expr} \rangle$ genera directamente sólo operadores + :

- $\langle \text{expr} \rangle$ es el símbolo de partida para expresiones

$\langle \text{term} \rangle$ genera operadores * :

- * va a aparecer siempre más abajo en el árbol, porque en cualquier derivación está más lejos del símbolo de partida que +

La gramática anterior genera el mismo lenguaje que las gramáticas de las diaps. 39 y 43, pero

- no es ambigua
- ... y especifica el orden de precedencia habitual de $+$ y $*$

P.ej., dos derivaciones para la sentencia $A = B + C * A$

Derivación *leftmost*

```

<assign> => <id> = <expr>
=> A = <expr>
=> A = <expr> + <term>
=> A = <term> + <term>
=> A = <factor> + <term>
=> A = <id> + <term>
=> A = B + <term>
=> A = B + <term> * <factor>
=> A = B + <factor> * <factor>
=> A = B + <id> * <factor>
=> A = B + C * <factor>
=> A = B + C * <id>
=> A = B + C * A
  
```

Derivación *rightmost*

```

<assign> => <id> = <expr>
=> <id> = <expr> + <term>
=> <id> = <expr> + <term> * <factor>
=> <id> = <expr> + <term> * <id>
=> <id> = <expr> + <term> * A
=> <id> = <expr> + <factor> * A
=> <id> = <expr> + <id> * A
=> <id> = <expr> + C * A
=> <id> = <term> + C * A
=> <id> = <factor> + C * A
=> <id> = <id> + C * A
=> <id> = B + C * A
=> A = B + C * A
  
```

La conexión entre árboles de *parse* y derivaciones es estrecha:

- cualquiera puede ser construida a partir de la otra

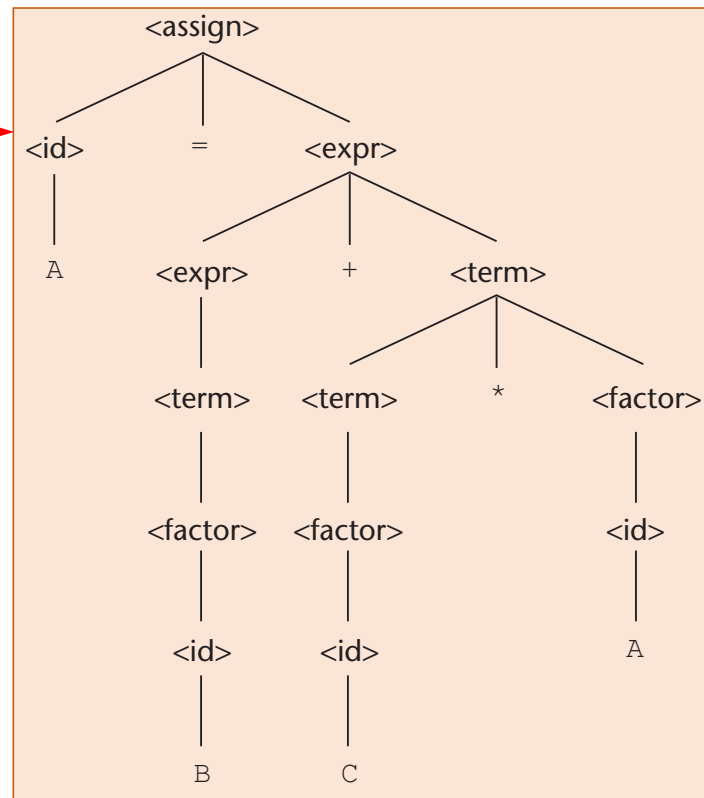
Toda derivación con una gramática no ambigua tiene un único árbol de *parse*
... aunque ese árbol puede ser representado por distintas derivaciones

La conexión entre árboles de *parse* y derivaciones es estrecha:

- cualquiera puede ser construida a partir de la otra

Toda derivación con una gramática no ambigua tiene un único árbol de *parse*
... aunque ese árbol puede ser representado por distintas derivaciones

Árbol de *parse* único para
 $A = B + C * A$
... usando la gramática no
ambigua de la diapos. 39

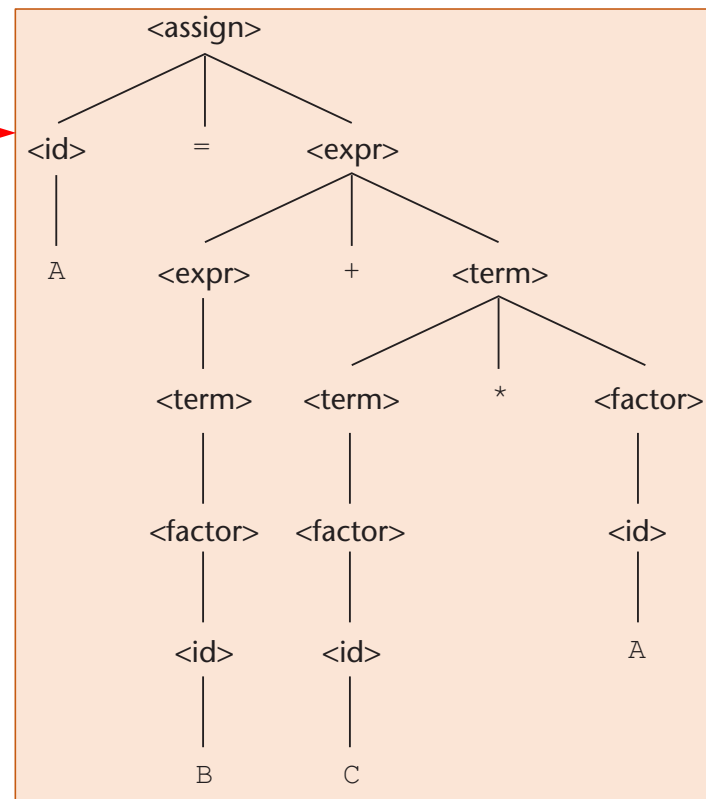


La conexión entre árboles de *parse* y derivaciones es estrecha:

- cualquiera puede ser construida a partir de la otra

Toda derivación con una gramática no ambigua tiene un único árbol de *parse* ... aunque ese árbol puede ser representado por distintas derivaciones

Árbol de *parse* único para
 $A = B + C * A$
 ... usando la gramática no ambigua de la diapos. 39



$\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$
 $\Rightarrow A = \langle \text{expr} \rangle$
 $\Rightarrow A = \langle \text{expr} \rangle + \langle \text{term} \rangle$
 $\Rightarrow A = \langle \text{term} \rangle + \langle \text{term} \rangle$
 $\Rightarrow A = \langle \text{factor} \rangle + \langle \text{term} \rangle$
 $\Rightarrow A = \langle \text{id} \rangle + \langle \text{term} \rangle$
 $\Rightarrow A = B + \langle \text{term} \rangle$
 $\Rightarrow A = B + \langle \text{term} \rangle * \langle \text{factor} \rangle$
 $\Rightarrow A = B + \langle \text{factor} \rangle * \langle \text{factor} \rangle$
 $\Rightarrow A = B + \langle \text{id} \rangle * \langle \text{factor} \rangle$
 $\Rightarrow A = B + C * \langle \text{factor} \rangle$
 $\Rightarrow A = B + C * \langle \text{id} \rangle$
 $\Rightarrow A = B + C * A$

$\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle + \langle \text{term} \rangle$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle + \langle \text{term} \rangle * \langle \text{factor} \rangle$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle + \langle \text{term} \rangle * \langle \text{id} \rangle$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle + \langle \text{term} \rangle * A$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle + \langle \text{factor} \rangle * A$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle + \langle \text{id} \rangle * A$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle + C * A$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{term} \rangle + C * A$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{factor} \rangle + C * A$
 $\Rightarrow \langle \text{id} \rangle = \langle \text{id} \rangle + C * A$
 $\Rightarrow \langle \text{id} \rangle = B + C * A$
 $\Rightarrow A = B + C * A$

Asociatividad es la regla semántica que especifica cuál operador tiene precedencia en expresiones con dos operadores con la misma precedencia:

- p.ej., $A / B * C$ o también $A / B / C$

Una gramática puede implicar correctamente la asociatividad de operadores

Asociatividad es la regla semántica que especifica cuál operador tiene precedencia en expresiones con dos operadores con la misma precedencia:

- p.ej., $A / B * C$ o también $A / B / C$

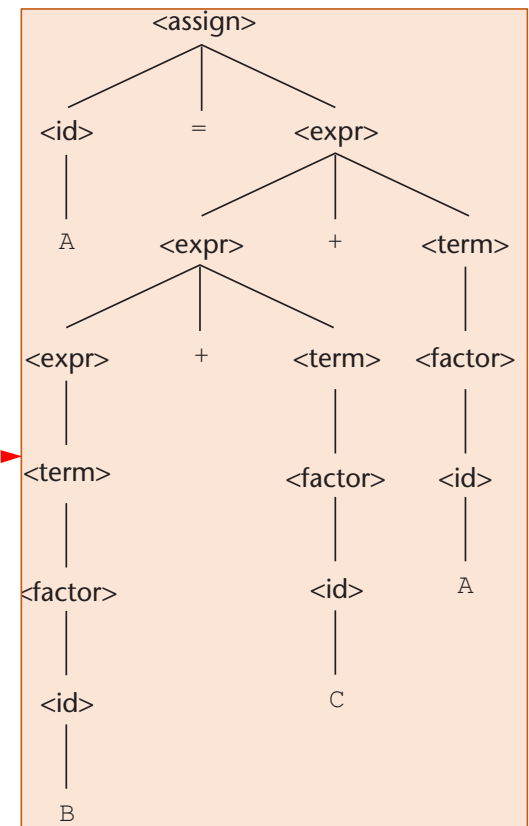
Una gramática puede implicar correctamente la asociatividad de operadores

P.ej., la gramática de la diap. 39 produce este árbol de *parse* para

$$A = B + C + A$$

... en que el $+$ de la izquierda aparece más abajo que el de la derecha:

- orden correcto si la suma es asociativa por la izquierda (típico en un lenguaje de programación)



En matemáticas, la suma es **asociativa**:

- *asociatividad por la izquierda y asociatividad por la derecha* producen el mismo resultado

... pero la resta y la división no lo son:

- una correcta asociatividad es esencial

$$(B + C) + A = B + (C + A)$$

En matemáticas, la suma es **asociativa**:

- *asociatividad por la izquierda y asociatividad por la derecha* producen el mismo resultado

... pero la resta y la división no los son:

- una correcta asociatividad es esencial

$$(B + C) + A = B + (C + A)$$

$$(B - C) - A \neq B - (C - A)$$

$$(B / C) / A \neq B / (C / A)$$

Reglas BNF para la
sentencia if-else en Java

```
<if_stmt> → if (<logic_expr>) <stmt>  
          | if (<logic_expr>) <stmt> else <stmt>
```

Reglas BNF para la
sentencia if-else en Java

```
<if_stmt> → if (<logic_expr>) <stmt>  
          | if (<logic_expr>) <stmt> else <stmt>
```

Si agregamos la regla

```
<stmt> → <if_stmt>
```

entonces la gramática se vuelve ambigua

Reglas BNF para la
sentencia **if-else** en Java

$\langle \text{if_stmt} \rangle \rightarrow \text{if } (\langle \text{logic_expr} \rangle) \langle \text{stmt} \rangle$
 $\quad \quad \quad | \text{if } (\langle \text{logic_expr} \rangle) \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle$

Si agregamos la regla

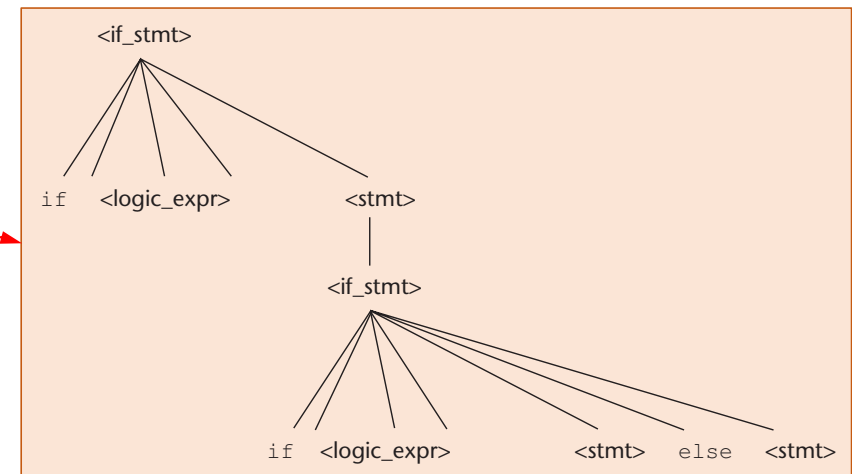
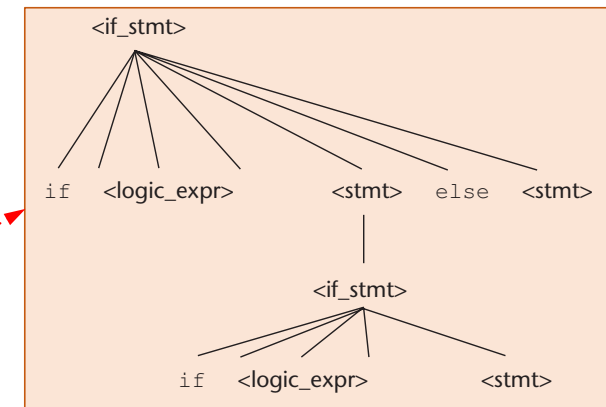
$\langle \text{stmt} \rangle \rightarrow \langle \text{if_stmt} \rangle$

entonces la gramática se vuelve ambigua

P.ej., la forma sentencial

if ($\langle \text{logic_expr} \rangle$) **if** ($\langle \text{logic_expr} \rangle$) $\langle \text{stmt} \rangle$ **else** $\langle \text{stmt} \rangle$

... tiene dos árboles de parse



Reglas BNF para la
sentencia **if-else** en Java

$\langle \text{if_stmt} \rangle \rightarrow \text{if } (\langle \text{logic_expr} \rangle) \langle \text{stmt} \rangle$
 $\quad \mid \text{if } (\langle \text{logic_expr} \rangle) \langle \text{stmt} \rangle \text{ else } \langle \text{stmt} \rangle$

Si agregamos la regla

$\langle \text{stmt} \rangle \rightarrow \langle \text{if_stmt} \rangle$

entonces la gramática se vuelve ambigua

P.ej., la forma sentencial

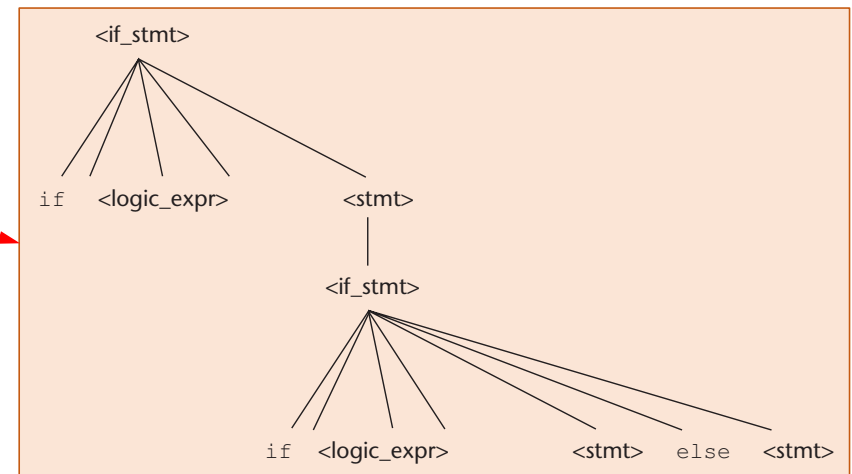
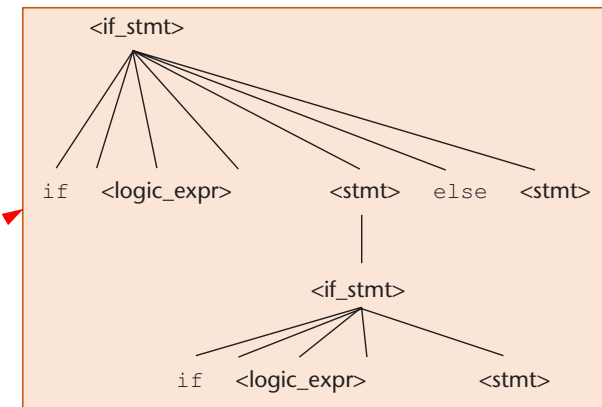
if ($\langle \text{logic_expr} \rangle$) **if** ($\langle \text{logic_expr} \rangle$) $\langle \text{stmt} \rangle$ **else** $\langle \text{stmt} \rangle$

... tiene dos árboles de parse

if (done == **true**)
if (denom == 0)
 quotient = 0;
else quotient = num / denom;

?

?



Gramática no ambigua para la sentencia **if-else** en Java

... basada en la práctica de que una cláusula *else*, si aparece, es emparejada con la cláusula *then* previa más cercana no emparejada

No puede ser un if sin else

$\langle \text{stmt} \rangle \rightarrow \langle \text{matched} \rangle \mid \langle \text{unmatched} \rangle$
 $\langle \text{matched} \rangle \rightarrow \text{if } (\langle \text{logic_expr} \rangle) \langle \text{matched} \rangle \text{ else } \langle \text{matched} \rangle$
 $\mid$ any non-if statement
 $\langle \text{unmatched} \rangle \rightarrow \text{if } (\langle \text{logic_expr} \rangle) \langle \text{stmt} \rangle$
 $\mid \text{if } (\langle \text{logic_expr} \rangle) \langle \text{matched} \rangle \text{ else } \langle \text{unmatched} \rangle$

Cálculo del MCD de dos enteros usando el algoritmo de Euclides

68

Lenguaje de máquina,
expresado como números
hexadecimales, para el
set de instrucciones x86

```
55 89 e5 53 83 ec 04 83 e4 f0 e8 31 00 00 00 89 c3 e8 2a 00
00 00 39 c3 74 10 8d b6 00 00 00 00 39 c3 7e 13 29 c3 39 c3
75 f6 89 1c 24 e8 6e 00 00 00 8b 5d fc c9 c3 29 d8 eb eb 90
```

Lenguaje assembly x86

```
pushl    %ebp
movl     %esp, %ebp
pushl    %ebx
subl     $4, %esp
andl     $-16, %esp
call     getint
movl     %eax, %ebx
call     getint
cmpl     %eax, %ebx
je       C
A: cmpl   %eax, %ebx
      jle  D
      subl %eax, %ebx
B: cmpl   %eax, %ebx
      jne  A
C: movl   %ebx, (%esp)
      call putint
      movl -4(%ebp), %ebx
      leave
      ret
D: subl   %ebx, %eax
      jmp  B
```

Cálculo del MCD de dos enteros usando el algoritmo de Euclides

69

Lenguaje imperativo:
C

```
int gcd(int a, int b) {  
    while (a != b) {  
        if (a > b) a = a - b;  
        else b = b - a;  
    }  
    return a;  
}
```

Lenguaje funcional:
OCaml

```
let rec gcd a b =  
    if a = b then a  
    else if a > b then gcd b (a - b)  
    else gcd a (b - a)
```

Lenguaje lógico:
Prolog

```
gcd(A,B,G) :- A = B, G = A.  
gcd(A,B,G) :- A > B, C is A-B, gcd(C,B,G).  
gcd(A,B,G) :- B > A, C is B-A, gcd(C,A,G).
```